

XML

En la primera unidad se ha introducido XML se ha trabajado con documentos XML básicos.

Recordemos que **XML** (*eXtensible Markup Language*) es un **lenguaje de marcas** utilizado para **representar y transmitir información** estructurada en un formato legible para las máquinas. Los documentos XML son similares a los documentos HTML, pero en lugar de ser utilizados para describir la presentación de la información, son utilizados para **describir la estructura y el contenido de la información**.

Un ejemplo de documento XML es el siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
</libro>
```



Estructura de un documento XML

Recordemos que las partes en las que se divide un documento XML son las siguientes:

- Un **prólogo**.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
```



- Un **ejemplar**.

```
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
</libro>
```



Definición de la estructura

El documento XML presentado a continuación está **incompleto** (aunque no incorrecto):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
</libro>
```



Está incompleto porque solo hemos declarado el tipo de documento que va a ser (`<!DOCTYPE libro>`), es decir, qué ejemplar vamos a definir. Más concretamente, estamos diciendo que el ejemplar será `libro`, pero no sabemos qué elementos y atributos tiene `libro`.

Para solucionar este problema, en esta unidad presentaremos dos **lenguajes** para definir la estructura de los documentos XML:

- DTD (Document Type Definition).
- XSD (XML Schema Definition).

DTD y XSD son dos lenguajes de esquemas utilizados para describir la estructura de un documento XML y garantizar su validez.

En esta unidad haremos un repaso del lenguaje XML, profundizando en algunos conceptos que no se trataron en la primera unidad. Posteriormente, hablaremos

de XSD, que es el lenguaje de esquemas más utilizado en la actualidad (por tanto, no profundizaremos en DTD, aunque se presentará una introducción básica).

Prólogo

El prólogo informa al **intérprete XML** de todos aquellos datos que necesita para realizar su trabajo. Es una parte **opcional**, es decir, un documento XML puede ser interpretado igualmente si no tiene prólogo.

INTÉRPRETE XML

Un **intérprete XML** es un software encargado de leer e interpretar un documento XML. También se puede conocer como **procesador** o **parser**.

Consta de dos partes:

- La declaración XML.

```
<?xml version="1.0" encoding="UTF-8"?>
```



- La declaración del tipo de documento. También llamada **DOCTYPE**.

```
<!DOCTYPE libro>
```



Declaración XML

Una declaración XML permite definir:

- La versión de XML que se utiliza.
- La codificación de caracteres empleada.
- Si es un documento autónomo o no.

En el caso de ser incluida, **debe ser la primera línea del documento**.

Una declaración XML es la siguiente: `<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>`

La declaración XML podrá contener hasta tres atributos, los cuales definen los siguientes aspectos:

Definición	Atributo	Opcional
Versión XML	version	No
Codificación	encoding	Sí
Autonomía	standalone	Sí

Versión de XML

El atributo version se utiliza para indicar la versión del lenguaje XML que se utiliza en el documento. La versión actualmente recomendada y utilizada es la 1.0, por lo que se utiliza comúnmente en la mayoría de los documentos XML.

```
version="1.0"
```



La versión 1.0 del lenguaje XML fue publicada en el año 1998 por el W3C (World Wide Web Consortium) y estableció las reglas básicas para la sintaxis, la estructura y la validez de los documentos XML. Incluyó características como **elementos**, **atributos**, **entidades**, y **reglas** para la construcción de documentos bien formados y válidos.

El uso de la versión 1.0 en los documentos XML permite:

- A los procesadores de XML saber qué reglas se deben seguir para procesar y validar correctamente el documento.
- A los desarrolladores saber qué características y reglas están disponibles en la versión específica del lenguaje que están utilizando.

Aunque existen versiones posteriores de XML, como la 1.1, no se utilizan ampliamente debido a que la versión 1.0 es suficientemente potente y estable

para cubrir la mayoría de las necesidades de los documentos XML.

Codificación de caracteres

El atributo `encoding` indica la **codificación de caracteres** utilizada en el documento. La codificación de caracteres es el conjunto de **reglas** utilizadas para representar un **juego de caracteres** en un formato que puede ser procesado por una computadora.

La codificación más utilizada en los documentos XML es UTF-8 (Unicode Transformation Format-8-bits). UTF-8 es una codificación de caracteres de longitud variable que **permite representar todos los caracteres Unicode**, lo que la hace adecuada para documentos que contienen una variedad de idiomas y símbolos. Además, es **compatible con ASCII**, lo que lo convierte en una elección popular para la mayoría de los documentos XML (para saber más sobre codificación de caracteres, puedes consultar [aquí](#) o [aquí](#)).

```
encoding="utf-8"
```



El uso de la codificación UTF-8 en los documentos XML permite a los procesadores de XML y a las aplicaciones saber cómo deben interpretar los caracteres en el documento y cómo deben procesarlos correctamente. Si no se especifica una codificación, los procesadores de XML pueden asumir que la codificación es UTF-8, pero es **recomendable especificarla explícitamente** para evitar confusiones.

El atributo `encoding` es **opcional**, pero se recomienda especificarlo para evitar confusiones y garantizar que el documento sea procesado correctamente.

Autonomía

El atributo `standalone` de una declaración XML es utilizado para indicar si el documento XML depende o no de una definición externa de entidades. La definición de entidades se pueden recoger en un documento DTD externo o en una sección interna del propio documento XML.

Hasta este momento, el atributo `standalone`, en el caso de tenerlo declarado, siempre ha sido `yes`, ya que los documentos generados eran independientes. Si

el valor del atributo `standalone` es `yes`, significa que el documento XML no depende de ninguna definición externa de entidades, es decir, el documento contiene toda la información necesaria para su procesamiento.

```
standalone="yes"
```



Por otro lado, si el valor del atributo `standalone` es `no`, significa que el documento XML depende de una definición externa de entidades, que debe ser proporcionado para validar y procesar el documento correctamente.

```
standalone="no"
```



El atributo `standalone` es opcional, y si no se proporciona, los procesadores de XML podrán asumir que el documento no tiene entidades externas. Este atributo es útil para indicar a los procesadores de XML si deben buscar una definición externa o no, lo que puede ahorrar tiempo y recursos.

Declaración del tipo de documento

La declaración del tipo de documento (`DOCTYPE`) en XML se utiliza para:

- Especificar el **tipo de documento XML**.
- Indicar la **ubicación de la definición** de tipo de documento (DTD).

La declaración `DOCTYPE` es **opcional**, pero es **recomendable** incluirla en los documentos XML para garantizar que el documento sea válido y para proporcionar información sobre el tipo de documento a los procesadores de XML.

Una declaración del tipo de documento es la siguiente:

```
<!DOCTYPE libro>
```



La declaración anterior está **incompleta**, ya que solo especifica el tipo de documento. Para que el `DOCTYPE` esté completo, se debe incluir la **definición del tipo de documento**.

La definición del tipo de documento viene definida en un **documento DTD**. Este documento puede estar definido de dos formas:

- En un **estándar público**. Por ejemplo, en estándares con **HTML** o **SVG**.
- **Localmente**.

❗ DOCUMENTO DTD

Un documento DTD permite, entre otros aspectos, definir:

- Los **tipos** de los **elementos** y **atributos** que se pueden utilizar en el documento XML.
- Las **restricciones** del documento.
- Los **valores por defecto**.

Estas características hacen que **los DTD permitan validar un documento XML**. Aunque en este apartado no se pretende conocer este tipo de documentos en detalle, sí se debe saber que contiene una serie de reglas que definen cómo debe ser el ejemplar del documento XML.

Definición pública

Cuando un DTD está definido en un estándar público, se utiliza un DOCTYPE como el siguiente:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN" "https://www.w3.org/TR/html401/"
```

A continuación del tipo de documento (html), se define:

- La palabra **PUBLIC**.
- Un **identificador público formal** o FPI (Formal Public Identifier).
- Opcionalmente, la **ubicación** del fichero DTD.

La información que podemos obtener del DOCTYPE anterior es la siguiente:

Elemento	Valor
----------	-------

Tipo de documento	html
Ámbito	PUBLIC
Identificador	-//W3C//DTD HTML 4.01//EN
Documento DTD	https://www.w3.org/TR/html4/strict.dtd

En este DOCTYPE se hace referencia a un fichero **strict.dtd**, el cual guarda las reglas de cómo se debe estructurar el documento HTML. Un documento HTML no deja de ser un XML con una serie de restricciones, las cuales se definen en el fichero DTD.

Definición local

Cuando un DTD no está definido en un estándar público, se utiliza un DOCTYPE como el siguiente:

```
<!DOCTYPE libro SYSTEM "libro.dtd">
```



Es el tipo de definición que se utilizará en esta unidad.

A continuación del del tipo de documento (libro), se define:

- La palabra `SYSTEM`.
- La **ubicación** del fichero DTD. Este fichero debe existir para que el documento XML se pueda procesar correctamente.

La información que podemos obtener del DOCTYPE anterior es la siguiente:

Elemento	Valor
Tipo de documento	libro
Ámbito	SYSTEM

Documento DTD

libro.dtd

Al igual que en el caos de la definición pública, en el DOCTYPE se hace referencia a un fichero DTD. En este caso, libro.dtd, el cual se debe ser un fichero definido en el sistema de ficheros.

Ejemplar

Es la **parte principal** de un documento XML, ya que contiene los **datos reales del documento**. Se ubica a **continuación del prólogo** y está formado por diferentes elementos anidados, los cuales guardan la información mediante el uso de etiquetas de marcado.

En el siguiente ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
</libro>
```



el ejemplar es:

```
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
</libro>
```



Algunas **características** de un ejemplar son:

- Todos los datos del documento XML que se deben procesar están definidos en el ejemplar.
- Tiene que tener un **único elemento raíz** (padre) del que desciendan todos los demás.
- Está compuesto de **elementos estructurados** según una estructura de árbol en la que el elemento raíz es el ejemplar y las hojas los elementos terminales, es decir, aquellos que no contienen elementos.
- Un elemento puede contener **texto, otro elemento o contenido mixto** (texto y otros elementos).
- Los elementos pueden tener **atributos asignados**.
- El nombre en la declaración de tipos del documento debe coincidir con el tipo de elemento del elemento raíz:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
</libro>
```



Es decir, el nombre libro definido en `<!DOCTYPE libro>` debe coincidir libro definido en `<libro>`.

Un documento XML está formado por texto plano (sin formato) y contiene marcas (etiquetas) definidas por el desarrollador.

Principalmente, un documento XML está formado por:

- Elementos.
- Atributos.
- Entidades. Los atributos y entidades no tienen por qué aparecer en un documento XML, pero los elementos sí. Por lo menos, tiene que haber un elemento (el elemento raíz), sino el documento XML no estaría bien formado.

Elementos

Los elementos son la estructura básica de un documento XML y representan un bloque de información. Un elemento se crea mediante etiquetas de marcado (marcas) y, opcionalmente, contenido.

```
<titulo>El señor de los anillos</titulo>
```



Sus características son:

- Se pueden crear elementos **vacíos**: `<etiqueta></etiqueta>` o `<etiqueta/>`. Es decir, se pueden crear elementos sin contenido.
- Un elemento (padre) puede contener a otro u otros elementos (hijos).
- Un elemento puede contener contenido mixto, es decir, texto y otros elementos.
- Todo documento XML tiene que tener un **único elemento raíz** (padre) del que desciendan todos los demás.
- La estructura de cualquier documento XML se puede representar como un **árbol invertido de elementos**.
- Los elementos son los que dan estructura semántica al documento.

Algunas normas básicas de sintaxis son:

- El **orden** de los elementos es **significativo**.
- Todos los nombres de los elementos son **sensibles a letras minúsculas y mayúsculas** (*case sensitive*).
- Pueden contener letras minúsculas, letras mayúsculas, números, puntos (.), guiones medios (-) y guiones bajos (_).
- Asimismo, pueden contener el carácter dos puntos (:). Pero, su uso se reserva para definir espacios de nombres.
- El primer carácter tiene que ser una letra o un guion bajo (_).
- Detrás del nombre de una etiqueta se permite escribir un espacio en blanco o un salto de línea.
- No puede haber un salto de línea o un espacio en blanco antes del nombre de una etiqueta.

- Las letras no inglesas (á, Á, ñ, Ñ, etc.) están permitidas. Pero, al igual que el carácter guion medio (-) y el punto (.), se recomienda no utilizarlos para reducir posibles incompatibilidades o errores en programas que no los interpreten bien.
- No puede comenzar por la cadena `xml`, ni ninguna de sus versiones en que se cambien mayúsculas y minúsculas (`XML`, `Xml`, `xML`, etc.).

Atributos

Los atributos añaden **información adicional** a los elementos.

```
<titulo idioma="es">El señor de los anillos</titulo>
```



Algunas normas básicas de sintaxis son:

- El orden de los atributos no es significativo.
- Los atributos se escriben dentro de la etiqueta de apertura del elemento.
- Los nombres de los atributos deben cumplir las mismas normas de sintaxis que los nombres de los elementos.
- En un elemento, los nombres de los atributos deben ser únicos. Es decir, no se pueden repetir nombres de atributos dentro de un mismo elemento.
- Los atributos de un elemento deben separarse con espacios en blanco.
- El valor de un atributo se delimita con comillas dobles " o comillas simples '.
- No se puede utilizar el carácter delimitador del valor de un atributo (" o ') como contenido. Es decir, si se utiliza " para encerrar el valor del atributo, no se puede utilizar " dentro del valor, aunque sí '.

! ELEMENTOS O ATRIBUTOS

En algunas ocasiones no queda claro cuándo almacenar información como contenido de un elemento o como valor de un atributo. A continuación, se muestran algunas **recomendaciones** de cuándo utilizar elementos y cuándo utilizar atributos.

Un dato deberá ser un **elemento** cuando:

- Contiene **subestructuras**.
- Es de un **tamaño** considerable.
- Su **valor cambia** frecuentemente.
- Su valor va a ser **mostrado** a un usuario/usuario o aplicación.

Un dato deberá ser un atributo cuando:

- El dato es de **pequeño tamaño** y su **valor raramente cambia**, aunque hay situaciones en las que este caso puede ser un elemento.
- El dato solo puede tener unos cuantos **valores fijos**.
- El dato **guía el procesamiento XML** pero no se va a mostrar.

Entidades

Las entidades son un mecanismo para incluir caracteres especiales.

```
<titulo idioma="es">El se&#241;or de los anillos</titulo>
```



Algunas normas básicas de sintaxis son:

- Las entidades se utilizan mediante una **referencia** al identificador de la entidad.
- Existen una entidades XML definidas. Para utilizarlas, se utilizan el símbolo **&**, seguido del identificador de la entidad y **;**.
- Se pueden escribir referencias de caracteres Unicode con los símbolos **&#**, seguidos del valor decimal o hexadecimal del carácter Unicode que se quiera representar y, finalmente, añadiendo el carácter punto y coma (**;**). El valor hexacimal debe ir precedido de una **x**.

A continuación, se muestran algunos **ejemplos de entidades de caracteres Unicode**:

Caracter	Unicode (decimal)	Entidad (decimal)	Unicode (hexadecimal)	Entidad (hexadecimal)
----------	----------------------	----------------------	--------------------------	--------------------------

ñ	241	ñ	F1	ñ
€	8364	€	20AC	€

Algunos ejemplos de entidades XML son:

Caracter reservado	Identificador	Entidad XML
<	lt	<
>	gt	>
"	quot	"
'	apos	'
&	amp	&

Los caracteres anteriores **no se pueden utilizar como contenido** de un elemento o como valor de un atributo. En su lugar, se deben utilizar sus entidades por los siguientes motivos:

- El carácter menor que < es problemático porque indica el comienzo de una etiqueta.
- El carácter ampersand & es problemático, ya que se utiliza para indicar el comienzo de una referencia a entidad.
- Uso de la comilla doble " y de la comilla simple ' en atributos:
 - `<dato caracter="comilla doble(")" />`
 - `<dato caracter='comilla simple(')' />`
- Los valores de atributos escritos entre comillas dobles " sí pueden contener al carácter comilla simple ' y a la inversa:
 - `<dato caracter="comilla simple(')" />`
 - `<dato caracter='comilla doble( ")"' />` En XML, los espacios de nombres (*namespaces*) son un mecanismo para **evitar conflictos de nombres entre elementos y atributos** de diferentes vocabularios o

esquemas, es decir, para evitar ambigüedades que podrían surgir en caso de que haya elementos o atributos con el mismo nombre. Esto ocurre cuando fusionamos varios ficheros XML en uno solo y se presentan elementos con el mismo nombre, pero con diferente significado.

Los *namespaces* asignan un **prefijo único** a un conjunto de elementos y atributos de un vocabulario específico, permitiendo que los elementos y atributos de diferentes vocabularios coexistan en el mismo documento sin causar conflictos. Además, también nos permite agrupar todos los elementos y atributos relacionados de una aplicación XML para que el software pueda reconocerlos con facilidad.

También es posible no indicar ningún prefijo. En ese caso, el namespace hace referencia a todos los elementos y atributos en el documento. Es decir, existe un vocabulario único para todo el documento XML.

Namespace con prefijo #

En el siguiente ejemplo se muestra cómo se declaran dos *namespaces*:

```
<?xml version="1.0" encoding="UTF-8"?>
<cartas
  xmlns:baraja="https://www.lmsgi.com/baraja"
  xmlns:restaurante="https://www.lmsgi.com/restaurante">

  <baraja:carta>
    <baraja:palo>Corazones</baraja:palo>
    <baraja:numero>7</baraja:numero>
  </baraja:carta>

  <restaurante:carta>
    <restaurante:carnes>
      <restaurante:carne restaurante:precio="12.95">Filete de ternera<
      <restaurante:carne restaurante:precio="13.60">Solomillo a la pir
    </restaurante:carnes>

    <restaurante:pescados>
      <restaurante:pescado restaurante:precio="16.20">Lenguado al hc
      <restaurante:pescado restaurante:precio="15.85">Merluza en sal
```

```
</restaurante:pescados>
</restaurante:carta>

</cartas>
```

En el documento anterior se están integrando dos tipos de cartas:

- La carta de una baraja, definido en `xmlns:baraja`.
- La carta de un restaurante, definido en `xmlns:restaurante`.

El nombre utilizado para los elementos (`carta`) es el mismo en ambos casos, pero no tienen el mismo significado, por lo que se ha decidido crear dos espacios de nombres o, lo que es lo mismo, dos vocabularios diferentes.

Para indicar que un elemento pertenece a un espacio de nombres, se debe utilizar el prefijo en la etiqueta:

```
<baraja:carta></baraja:carta>
```



```
<restaurante:carta></restaurante:carta>
```



Ocurre lo mismo con los atributos:

```
<restaurante:carne restaurante:precio="12.95">Filete de ternera
```



Namespace sin prefijo

También es posible no indicar ningún prefijo:

```
<?xml version="1.0" encoding="UTF-8"?>
<baraja xmlns="https://www.lmsgi.com/baraja">
  <carta>
    <palo>Corazones</palo>
    <numero>7</numero>
```




```
</carta>  
</baraja>
```

Veremos que en XSD se utilizará la declaración de namespaces tanto con prefijo como sin prefijo.

Validación

XML es un lenguaje con una sintaxis muy sencilla, pero dotado de un importante y muy potente conjunto de herramientas. Entre las más destacas están las destinadas a garantizar la **correcta estructura de los documentos**. XML permite determinar las reglas que debe cumplir un determinado documento escrito en este lenguaje. Estas reglas, junto con los correspondientes procesadores XML, permiten garantizar que un documento y los datos almacenados en él están correctamente contruidos.

Una de las características de un metalenguaje como XML es que **no tiene etiquetas predeterminadas**. Esto significa que, en lugar de tener un listado de etiquetas con sus significados XML, proporciona una infraestructura sobre la cual crear etiquetas y dotarlas de significado.

No obstante, hay ciertas reglas generales de diseño que hay que cumplir y que dotan de robustez a los lenguajes creados con XML:

- El documento tiene que **estar bien formado** o, lo que es lo mismo, **respetar la sintaxis de XML**.
- El documento tiene que **ser válido** o, lo que es lo mismo, conforme a las **reglas definidas en su DTD o XML Schema**, aunque estas definiciones semánticas son opcionales.

Un documento XML tiene que ser forzosamente bien formado para que pueda ser interpretado por un procesador XML, pero no tiene por qué ser válido. Dicho de otro modo, un documento debe estar bien formado obligatoriamente, pero **es opcional que sea válido**.

XML bien formado

Un documento XML bien formado es un documento que **cumple con las reglas sintácticas y estructurales del lenguaje XML**. Un documento XML bien formado debe cumplir con las siguientes características:

- En el caso de definirla, la declaración XML debe realizarse en la primera línea del documento y debe ser válida.
- Debe existir un único elemento raíz, que contiene todos los demás elementos del documento.
- Todos los elementos deben estar cerrados, o bien con una etiqueta de cierre o cerrándose en la de apertura (elementos vacíos).
- Los elementos deben estar anidados correctamente, es decir, un elemento debe cerrarse antes de que se abra otro elemento.
- Los atributos deben estar dentro de comillas simples o dobles.
- Los caracteres especiales deben representarse utilizando entidades.
- En el caso de ser usados, los namespaces se deben definir y utilizar correctamente.

XML válido #

Un documento XML válido es aquel que:

- Está bien formado.
- Cumple con las restricciones especificadas en un DTD o XML Schema.

Es decir, **si un documento XML es válido, por definición, también estará bien formado.**

Los documentos XML válidos son más estrictos en cuanto a su contenido y estructura, y suelen ser requeridos para aplicaciones específicas o intercambio de datos.

En la primera unidad hemos visto cómo crear documentos XML bien formados. En esta, veremos cómo crear documentos XML válidos.